

Camunda 7 Exit Plan

We are running two end-of-life foundations in production. This sets out what it takes to land on a supported stack, and the one question only the reviewer can answer.

CAMUNDA 7.23.0 · EOL SPRING BOOT 3.5.16 · EOL DECISION REQUIRED

PART 1 – DECISION MEMO

1 Where we stand

bravo-bpm-service orchestrates loan origination across roughly thirty microservices. It runs on two foundations that are both past end of life.

FOUNDATION	WE RUN	STATUS	LAST PATCH
Camunda 7 Community Edition	7.23.0	Community Edition ended at 7.24.0; upstream repository archived	2025-10-14
Spring Boot	3.5.16	OSS support ended 2026-06-30 ; 3.5.16 was the final OSS patch	2026-06-25

Neither of these was a decision. Camunda 7 went end-of-life underneath us, and Spring Boot 3.5 reached its published cut-off date. We receive no security patches for either, and we sit one minor version behind even the final Community Edition release.

This matters more here than it would elsewhere. This service is the system of record for loan applications in flight at a regulated lender. An unpatched engine is not an abstract risk on a component of that kind.

The binding constraint is Spring Boot, not Camunda. That changes the question from “*should we replace Camunda?*” to “*what is the single*

migration that lands us on a supported stack?”

2 The constraint that shapes everything

LOAD-BEARING

Camunda 7.23's Spring Boot starter will not run on Spring Boot 4.

The coupling therefore runs in exactly one direction:

- Moving to Spring Boot 4 **forces** a decision about the engine.
- Swapping the engine does **not** force Spring Boot 4.

The consequence removes the most intuitive plan from the table: **“do the Spring Boot upgrade first and decide about Camunda later” is not available to us.** Any Spring Boot 4 work is blocked behind an engine decision.

The reverse is available — we could swap the engine and stay on Spring Boot 3 — but as Section 3 shows, that leaves us on an end-of-life Spring Boot regardless of which fork we choose.

3 The options

Camunda 7 has several community forks. Two are credible for a service of this criticality: **Operaton** and **CIB seven**. Both are Apache 2.0. Both keep the ACT_ database schema, accept the legacy camunda: BPMN namespace, and publish an automated OpenRewrite migration recipe.

Camunda 8 is **not** an upgrade path and is out of scope. It removes the embedded engine, replaces all 272 of our JavaDelegate classes with external job workers, drops the ACT_ tables from our database, and requires a paid licence for self-managed production use. It is a multi-quarter rewrite programme with its own budget line, not a version bump.

	A — SEQUENCED	B1 — COMBINED, OPERATON	B2 — COMBINED, CIB SEVEN
Engine	CIB seven 2.2.0	Operaton 2.1.4	CIB seven 2.2.0 (-4)

	A — SEQUENCED	B1 — COMBINED, OPERATON	B2 — COMBINED, CIB SEVEN
Spring Boot	3.5.14 · BACKWARDS	4.0.8	4.0.6
Camunda 7.24 hop first	not needed	required	not needed
Regression cycles	two	one	one
Ends on a supported stack	NO engine yes, Spring Boot no	YES	YES

Approach A is the only way to separate the two migrations in time, and CIB seven is the only fork that permits it, because it is the only one still shipping a Spring Boot 3 line. Its honest value is spreading risk across two smaller changes rather than one large one.

Its cost is real: it pays the full regression bill twice, it moves Spring Boot *backwards* from 3.5.16 to 3.5.14, and it leaves us on an end-of-life Spring Boot for however long the second step takes. It is a way-station, not a destination.

Approaches B1 and B2 are the same shape and differ only in which fork we adopt. Both land on a supported engine and a supported Spring Boot in a single regression cycle.

4 Recommendation

RECOMMENDED

Adopt Path B: migrate the engine and Spring Boot together, in one planned change.

The argument is the one-way coupling in Section 2. Because Spring Boot 4 work cannot proceed without an engine decision, sequencing does not let us defer the hard question — it only delays the benefit while paying the regression cost twice. And because Path A still leaves us on an end-of-life Spring Boot, it does not actually resolve the exposure that makes this urgent.

On which fork, this document deliberately does not decide. The technical evidence is laid out below; the deciding question is one only the reviewer can answer.

The two forks, on measured evidence

	OPERATON 2.1.4	CIB SEVEN 2.2.0
Commits, last 52 weeks	2,310	258
Distinct authors, 12 months	52	31
Top-contributor concentration	~80% of non-bot commits	25%
Ex-Camunda core engineers active	none	two
Long-term support	NONE 6-month lines	PAID terms undisclosed
Legal entity	none yet; non-profit planned	CIB software GmbH (Munich)
Requires 7.24 hop first	YES	NO
Ships process-test-coverage	NO 4 tests affected	YES
Ships a mockito helper	no (we do not use it)	yes
Keycloak identity provider	2.1.0 — lags the engine	2.2.0
engine.impl.util.CollectionUtil	ABSENT breaks 2 files	PRESENT
Cockpit webjar path	moves to webjars/operaton/	UNCHANGED no relocation

Read that table honestly and it says two different things at once.

CIB seven is cheaper to migrate to. It avoids five concrete frictions: the 7.24 hop; the CollectionUtil compile break; relocating our seven Cockpit plugin bundles; and the two missing test dependencies, process-test-coverage and mockito. Its bus factor is far healthier, it employs two engineers who wrote large parts of the original engine, and a support contract is at least theoretically purchasable.

Operaton is the more vital project. Nine times the commit volume, two-thirds more distinct contributors, a broader ecosystem under one organisation, and an explicit commitment to Apache 2.0 with no paid tier and no open core.

The question that decides it

Do we require a contractual support agreement for the workflow engine, and can one actually be purchased for a BFI entity?

If yes → CIB seven. Company-backed, deep original-engine expertise, cheaper migration, and a purchasable relationship.

If no → Operaton. Over a five-to-ten-year horizon on a core engine, sustained project activity is worth more than a one-off migration saving, and Operaton leads on that by a wide margin.

UNRESOLVED

CIB seven's actual support terms are **not published** — length, service levels and pricing are all sales-contact-only. If the reviewer leans that way, that enquiry should start before the decision is finalised, not after.

A note on risk posture

We are already running two end-of-life foundations in production, and no control has stopped us. That tells us there is no hard, enforced gate here — but two situations that look similar are worth distinguishing:

- What we have **now** is drift. Camunda 7 went end-of-life underneath us.
- What we would be **choosing** is a project with a six-month support window, a concentrated maintainer base, and (for Operaton) no legal entity yet.

Those attract different scrutiny if anyone ever examines them. Today's silence should not be read as pre-approval for tomorrow's deliberate choice.

The counterweight is equally real, and it argues for moving rather than waiting: **Camunda 7 Community Edition had the long support window, and it still ended in an archived repository with no upgrade path.** A six-month line with monthly patches is, in practice,

more responsive to security issues than a multi-year line that stops dead. The choice is not between a treadmill and stability. It is between a treadmill we control and a cliff we do not see coming — and we have just been over the cliff.

5 What is being asked of the reviewer

- 1 **Approve Path B** — engine and Spring Boot migrated together — or direct us to Path A.
- 2 **Choose the fork**, or answer the support-agreement question in Section 4 so the choice follows from it.
- 3 **Greenlight the pre-decision cleanup in Section 7**, which is independent of both answers and can start immediately.

Items 1 and 2 gate the migration. Item 3 does not, and delaying it costs us a smaller, messier diff later.

PART 2 — EXECUTION

6 Pre-flight gates

These run **before** any data is committed. Two of them can change the plan.

6.1 Schema-version reconciliation — the one real database risk

Our two Flyway migrations add four columns and two indexes to Camunda-owned tables:

```
V2_0_202412300429__alter-camunda-table.sql  
V2_0_202412301022__alter-camunda-table-add-index.sql
```

These turn out to be **verbatim replays of Camunda's own upgrade DDL** — the 7.20 → 7.21 and 7.21 → 7.22 scripts shipped inside the engine jar add exactly these four columns and these two indexes, matching index names included.

INFERENCE — STRONG, NOT VERIFIED

Our database was never engine-upgraded through the 7.21 and 7.22 steps. Someone hand-replayed the official DDL through Flyway instead. The columns arrived; the engine's own record of *what version the schema is at* may not have moved with them.

That matters because engine upgrade scripts use bare `add column` with **no IF NOT EXISTS** — unlike our defensive Flyway copies. If `ACT_GE_SCHEMA_LOG` reports a version below 7.22 while those columns already exist, the engine will attempt an upgrade that cannot succeed, and startup fails with `column "batch_id_" of relation "act_ru_job" already exists`.

Our `IF NOT EXISTS` guards made the migrations idempotent, which was correct — and also made the drift silent, because nothing ever failed loudly enough to tell us.

RUN SEPARATELY PER ENVIRONMENT

Run against SIT, UAT and PROD independently. **Do not infer PROD from SIT.**

```
-- 1. What version does the engine believe the schema is at?
SELECT id_, version_, timestamp_ FROM act_ge_schema_log
ORDER BY CAST(id_ AS INTEGER) DESC LIMIT 10;

-- 2. Engine-recorded properties
SELECT name_, value_ FROM act_ge_property;

-- 3. Do the four hand-added columns exist?
SELECT table_name, column_name, data_type FROM information_schema.columns
WHERE (table_name, column_name) IN (
    ('act_ru_job', 'root_proc_inst_id'), ('act_ru_job', 'batch_id'),
    ('act_hi_job_log', 'batch_id'), ('act_hi_procinst', 'restarted_proc_inst_id'))
ORDER BY 1,2;

-- 4. Are both indexes present?
SELECT indexname, tablename FROM pg_indexes WHERE indexname IN
    ('act_idx_job_root_procinst', 'act_idx_hi_pro_rst_pro_inst_id');
```

RESULT	MEANING	ACTION
Top row 1300 / 7.24.0 , all four columns present	CLEAR schema and label agree	No DDL runs. Proceed.
Version below 7.22, columns present	DRIFT confirmed	Insert the missing ACT_GE_SCHEMA_LOG rows to reconcile the label before cutover
Version at or above 7.22, columns missing	INVERSE drift	Investigate — not expected, but would invalidate assumptions

For reference, the entire Camunda 7.23 → 7.24 Postgres upgrade script is one row:

```
insert into ACT_GE_SCHEMA_LOG values ('1300', CURRENT_TIMESTAMP, '7.24.0');
```

Zero DDL. The documented 7.24 prerequisite costs nothing at the database layer.

6.2 Process-variable census

In-flight variables are expected to be safe: nothing in `src/main/java` uses Spin or the `ObjectValue` API, and stored values are `String`, `Boolean`, `Double` and `java.util.UUID` — a JDK class untouched by the package rename. Confirm against real data:

```
SELECT type_, COUNT(*) FROM act_ru_variable GROUP BY type_ ORDER BY 2 DESC;
```

Any `serializable` rows carrying an `org.camunda.*` class name would be the single blocker to a hot cutover. No code path was found that creates one.

6.3 Production-clone rehearsal

Non-negotiable. Restore a PROD clone, run the full cutover against it, and verify:

- the engine starts and applies no unexpected DDL
- in-flight instances resume and advance
- user tasks remain claimable with existing assignments
- the Cockpit plugins load and render
- pending jobs and timers fire

WHERE TO SPEND THE REHEARSAL BUDGET

Running two engine versions against the same ACT_RU_JOB table during a rolling deployment is the one genuinely untested area of this migration.

7 Work that lands now, independent of the decision

Six items. None depends on which path or fork is chosen; all shrink the eventual migration diff. Four are pure deletion.

#	CHANGE	WHY NOW	RISK
1	Replace <code>org.camunda.bpm.engine.impl.util.CollectionUtil</code> with <code>org.springframework.util.CollectionUtils</code> in <code>SurveyorCoverageProduct.java</code> and <code>SurveyorCoverageProductDto.java</code> (6 call sites)	This class is absent from Operaton entirely ; the migration recipe would rename the import to a class that does not exist and break the compile. Both files already import Spring's version and use it four lines away for the identical check.	NONE behaviourally identical
2	Delete <code>instance-tab-modify.js</code> (219 KB)	Commented out in <code>config.js</code> and the commented reference misspells it <code>.hjs</code> , so it has never loaded	NONE dead code
3	Remove <code>camunda-bpm-mockito</code> from <code>pom.xml</code>	Declared; imported by zero test files	NONE
4	Remove <code>com.vladmihalcea:hibernate-types-55</code>	Declared; imported by zero files	NONE
5	Remove the OpenTracing / Jaeger stack and <code>config/TraceConfig.java</code>	Both upstream projects are retired; the pinned versions are already the last ever published. The sole consumer returns a no-op tracer behind a condition that is normally false.	SIGN-OFF removes a tracing hook, even a dormant one

#	CHANGE	WHY NOW	RISK
6	Delete config/OldSecurityConfig.java a	Entirely commented-out KeycloakWebSecurityConfigurerAdapt pter	NONE

Items 1 and 2 are migration-relevant. Items 3–6 are hygiene that happens to reduce the Spring Boot 4 surface.

8 Path B — combined migration

Shared spine

1. **Pre-flight gates pass** (Section 6). Reconcile schema-log drift if found.
2. **Pre-decision cleanup landed** (Section 7).
3. **Branch, then run the migration recipe.** Mechanical rename across 687 files (351 main, 336 test), dependency coordinates and property keys.
4. **Resolve recipe fallout** — the known gaps are listed per-fork below.
5. **Spring Boot 3.5.16 → 4.0.x.** Target 4.0.x, **not** 4.1.x: `spring-cloud` Oakwood (2025.1.x) is the only GA release train and it pins `spring-boot.version` to 4.0.8.
6. **Full regression** — 1,457 test files, including the 12 test classes in the `functional/` package and the BPMN parse suite.
7. **Deploy** per Section 13.

Java 17 remains valid throughout. Spring Boot 4 supports JDK 17–26 and both forks build against Java 17, so no JDK change is required.

Spring Boot 4 work, independent of the engine

This is the larger half and we owe it regardless:

DRIVER	DETAIL
@MockBean → @MockitoBean	308 occurrences across 186 files — removed in Boot 4. The dominant mechanical change.
Hibernate 6 → 7	via <code>hypersistence-utils-hibernate-63</code> → <code>-70</code> (artifactId change); 325 JsonBinaryType occurrences across 86 files , against 238

DRIVER	DETAIL
	@Entity classes
ShedLock 4.42 → 7.9	three majors; 11 @SchedulerLock sites — the 10 schedulers plus AppStartupRunner
Remove spring-framework-bom pin (6.2.19)	Would silently fight Boot 4's own BOM. The single most dangerous pin in pom.xml .
Spring Security 6 → 7, Spring Data 3 → 4, Tomcat 10.1 → 11	remove the explicit pins; let the Boot 4 BOM manage them
springdoc-openapi 2.8.11 → 3.1.1	A major bump — its POM targets Boot 4 module names
javax.* leftovers	4 files, trivial

Fork-specific deltas

	B1 — OPERATON 2.1.4	B2 — CIB SEVEN 2.2.0
Prerequisite	Bump 7.23.0 → 7.24.0 first (documented; zero DDL)	none
Recipe	org.operaton:migrate-camunda-recipe:1.0.1 (Maven artifact)	org.cibseven.ReplaceCamundaWithCIBSeven (hosted YAML, not a Maven artifact)
Property prefix	camunda.bpm.* → operaton.bpm.* ; sub-keys unchanged	UNVERIFIED confirm during the spike
Cockpit bundles	relocate webjars/camunda/ → webjars/operaton/	NO RELOCATION path unchanged
Session include-path	update /bpm/camunda, /camunda → Operaton equivalent	expected unchanged; confirm
CollectionUtil	absent — resolved by Section 7 item 1	present
Process-test-coverage	NO PORT 4 functional tests need rework	ships cibseven-process-test-coverage-junit5-platform-7

	B1 — OPERATON 2.1.4	B2 — CIB SEVEN 2.2.0
Keycloak plugin	operaton-keycloak 2.1.0 (lags engine 2.1.4)	cibseven-keycloak 2.2.0

VERIFIED FOR BOTH FORKS

The Cockpit `config.js` `contract( customScripts , requireJsConfig , bpmnJs.additionalModules )` is preserved, and all four plugin registration points our bundles use are present in each fork's cockpit bootstrap.

9 Path A — sequenced, via CIB seven

Only viable on CIB seven, the sole fork still shipping a Spring Boot 3 line.

Step 1 (now): Camunda 7.23.0 → CIB seven 2.2.0 on Spring Boot 3.5.14. Engine-only change; no Spring Boot 4 work, no `@MockBean` migration, no Hibernate 7.

Step 2 (later): CIB seven 2.2.0 → the `-4` artifacts on Spring Boot 4.0.6. Spring Boot only; the engine does not move again.

What it buys: two smaller changes instead of one large one, each with its own rollback point and its own regression cycle. Genuinely lower per-change risk.

What it costs:

- Spring Boot moves **backwards**, 3.5.16 → 3.5.14, and 3.5.16 was the final OSS patch
- We remain on an end-of-life Spring Boot for the entire gap between the steps
- The full regression bill is paid **twice**
- It commits us to CIB seven before the fork question has been properly weighed, because no other fork offers this route

Assessment: defensible only if the organisation cannot absorb a single change of Path B's size. It does not resolve the exposure that makes this urgent; it defers it.

10 Rollback

The rollback story is unusually good, and it rests on one fact: **both forks keep Camunda 7.24's database schema unchanged**, ACT_ prefix included. Operaton seeds ACT_GE_SCHEMA_LOG with '7.24.0' and terminates its upgrade chain at the same id '1300' Camunda uses. Neither fork renumbers.

STAGE	ROLLBACK	NOTES
Pre-decision cleanup (§7)	Ordinary revert	No engine involvement
7.24 bump (B1 only)	Redeploy 7.23 artifact	Schema-log row is additive and harmless
Recipe run, pre-deploy	Discard the branch	No production impact
SIT / UAT	Redeploy previous artifact	Standard
PROD cutover	Redeploy the previous artifact against the same database	Viable <i>because</i> no DDL runs. This is the load-bearing claim and must be proven in the §6.3 rehearsal, not assumed.

The one thing that does not roll back cleanly is the schema-log reconciliation in §6.1, if it proves necessary. Those inserted rows correct a real drift and should stay. Take a database backup immediately before, as standard practice.

Point of no return: none at the database layer, which is the unusual and welcome part. The practical point of no return is organisational — once PROD has run on the new engine long enough to accumulate history, reverting means running an unpatched engine again, not a data problem.

11 Blast radius

SURFACE	SIZE	EFFECT
Files importing org.camunda	687 351 main + 336 test	Mechanically renamed by the recipe
Delegate classes	272 225 + 47	JavaDelegate and BaseActivity.Imports change; logic untouched

SURFACE	SIZE	EFFECT
BPMN files	53	UNCHANGED legacy camunda: namespace accepted
DMN files	3	UNCHANGED
camunda:delegateExpression in BPMN	483	UNCHANGED resolve as before
Flyway migrations touching ACT_	2	UNCHANGED
Internal engine.impl imports	4 across 3 files	Context and BpmnExecutionContext resolve as-is; both CollectionUtil imports are fixed by §7 item 1
Cockpit / Tasklist plugin bundles	7 ~4.9 MB	Relocated (Operaton) or untouched (CIB seven)
@MockBean sites	308 across 186 files	Path B only
Total test files	1,457	Full regression required

Despite 687 touched files, the coupling is **shallow** — overwhelmingly public API. `engine.delegate` alone accounts for **531 of the main-source imports**, and 813 across main and test together. That is precisely the profile an automated recipe handles well.

12 Effort

WORK	ESTIMATE	CONFIDENCE
Pre-decision cleanup (§7)	1–2 days	HIGH
Spring Boot 4 alone	12–20 days	MEDIUM variance is Hibernate 7 and test fallout
Engine swap, CIB seven	+3–5 days	MEDIUM

WORK	ESTIMATE	CONFIDENCE
Engine swap, Operaton	+5–8 days	MEDIUM adds 7.24 hop, bundle relocation, test-coverage rework
PROD-clone rehearsal	2–3 days	MEDIUM

PATH	TOTAL	CYCLES
B2 — CIB seven combined	18–30 days	one
B1 — Operaton combined	20–33 days	one
A — sequenced	25–40 days	two

One engineer, excluding review, deployment soak and any work arising from the open questions in Section 15. Confidence on the inventory is high — every count was measured against the tree. Confidence on the day estimates is medium.

13 Deployment sequencing

Standard promotion: **SIT** → **soak** → **UAT** → **soak** → **PROD**, with the §6.1 diagnostics run independently against each environment first, since their schema-log state may differ.

CUTOVER SHAPE

A rolling restart, not a drain. Draining is not available to us in any case: 192 `userTask` elements across 26 of the 53 BPMN files mean instances park on human action indefinitely. In-flight instances resume because deployed BPMN re-parses from `ACT_GE_BYTEARRAY` — verified at bytecode level that both forks register the legacy `camunda:` namespace alongside their own.

Two conditions attach:

- Quiesce the job executor during the swap.** Two engine versions competing for the same `ACT_RU_JOB` rows is the untested area.
- Rehearse on a PROD clone first** (§6.3).

Configuration and Helm changes

ITEM	WHERE	CHANGE
Engine property prefix	application.yaml:621	camunda.bpm → operaton.bpm (B1). CIB SEVEN UNVERIFIED
History-cleanup keys	same block	Sub-keys carry over unchanged under the new prefix
Session include-path	application.yaml:3241	/bpm/camunda, /camunda → add the Operaton path (B1). Expected unchanged for B2
Webapp URL path	Helm ingress / probes	/camunda/app/. . . → Operaton equivalent (B1). UNVERIFIED FOR B2
Environment variable names	Helm values	NO CHANGE CAMUNDA_HISTORY_* are our own placeholders, not engine-defined

DELIBERATE OMISSION

Do not rename the environment variables as part of this migration. They are our own naming, the deployed values are what matter, and touching them multiplies the number of things that can go wrong at cutover for no functional gain.

14 What we lose or defer

Lost, on Operaton only: BPMN path-coverage HTML reports on four functional tests — DMNFunctionalTest , UnsecuredLMSIntegrationFunctionalTest , AdvanceAIKYCActivityFunctionalTest , AntiFraudEngineCheckpointFunctionalTest . Test *correctness* is unaffected; only the coverage reporting goes. CIB seven ships the library and loses nothing.

LATENT, BOTH PATHS

The seven Cockpit plugin bundles are committed as **built artefacts whose TypeScript sources are not in this repository.** rollup.config.mjs references a src/ directory that does not exist here, and there is no package.json . They will survive this migration untouched, because both forks preserve the plugin contract. They cannot survive a future change that alters it, because we cannot rebuild them.

Locating those sources is worth doing on its own merits, independent of this migration.

Deferred deliberately:

- Operaton's webapp-neo and CIB seven's webclient — both ship *alongside* the classic webapp rather than replacing it. No action needed now.
- Camunda 8 — a separate programme, not an upgrade (Section 3).
- Spring Boot 4.1 — blocked until a spring-cloud train targets it.

15 Open questions

#	QUESTION	BLOCKS	OWNER
1	Is a support agreement required, and is one purchasable for a BFI entity?	THE FORK DECISION	Architect / procurement
2	Are these six Cockpit capabilities in active use — historic activities on definitions and on instances, route history, auto-refresh, the bpmn-js robot module, and the Tasklist audit log?	The fallback option of dropping the plugins	Operations
3	What does ACT_GE_SCHEMA_LOG report in SIT, UAT and PROD?	The cutover plan	Engineering — SQL in §6.1
4	CIB seven's property prefix and webapp URL path	Accurate B2 estimate	Engineering — spike
5	What deployment window is actually available?	Cutover design	Release management

Question 1 is the critical path. Questions 3 and 4 are answerable in hours.

Sources. All version and artefact facts verified against repo1.maven.org directory listings and release-tag POMs on 2026-09-09; project health via the GitHub API on the same date;

schema and namespace behaviour verified by extracting and inspecting the published engine and webapp jars; all repository counts measured directly against the working tree.

Prior research, including the wider fork landscape and the Camunda 8 assessment, lives alongside this document in the repository.

Two corrections to that earlier research, both carried into this document: it states that CIB seven has no OpenRewrite migration recipe — CIB seven ships one as a hosted YAML configuration rather than as a Maven artifact, which is why a Maven-only search does not find it. It also counts the internal-API exposure as four files; it is four imports across three files.